



DEPARTMENT OF
COMPUTER SCIENCE



ARRAYS AND RECURSION I

DR JEREMIAH O. BANDELE

PhD Electrical and Electronic Engineering
University of Nottingham



LEARNING OBJECTIVES

At the end of the lesson, you should be able to:

- Describe the meaning, declaration, definition and initialisation of arrays.
- Discuss multi-dimensional arrays.
- Demonstrate a recursive process.



ARRAYS

An array is a type of data structure that can store a fixed-size sequential collection of elements of the same type.

Why do we need arrays?



ARRAYS: DECLARATION

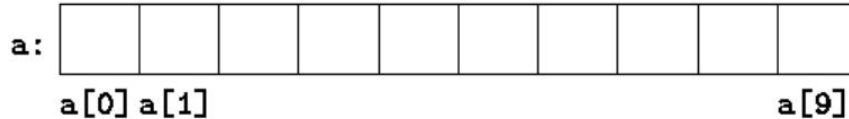
To declare an array in C, a programmer specifies the type of the elements and the number of elements required by an array

```
datatype arrayName [ arraySize ];
```

The declaration

```
int a[10];
```

defines an array of size 10, that is, a block of 10





ARRAYS: INITIALISATION

You can initialize an array in C either one by one or using a single statement as follows

```
double salaries[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

The number of values between braces { } cannot be larger than the number of elements that we declare for the array between square brackets []



ARRAYS: INITIALISATION

If you omit the size of the array, an array just big enough to hold the initialization is created.

```
double salaries[] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

How does this change in multidimensional arrays?



ARRAYS: INDEXING

An element is accessed by indexing the array name.

```
double salaries[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

```
double salary = salaries[2];
```

What is the value of salary?



ARRAYS: EXAMPLE

Array = (1,2,3,2,5,6,3,8,1,10)

Write a program in C to find the sum of all elements of the array above.



ARRAYS: EXAMPLE

Array = (1,2,3,2,5,6,3,8,1,10)

Write a program in C to copy the elements of the array above into another array



MULTI-DIMENSIONAL ARRAYS

A multi-dimensional array is defined using multiple adjacent square brackets, and the elements of the array may be initialised with values enclosed in curly braces.

```
float matrix[3][4] = {  
    { 2.4, 8.7, 9.5, 2.3 },  
    { 6.2, 4.8, 5.1, 8.9 },  
    { 7.2, 1.6, 4.4, 3.6 }  
};
```

```
float matrix[][4] = {  
    { 2.4, 8.7, 9.5, 2.3 },  
    { 6.2, 4.8, 5.1, 8.9 }  
};
```



MULTI-DIMENSIONAL ARRAYS: INDEXING

To access an element of a multidimensional array, the correct notation is to enclose each subscript in a separate pair of square braces.

```
float matrix[3][4] = {  
    { 2.4, 8.7, 9.5, 2.3 },  
    { 6.2, 4.8, 5.1, 8.9 },  
    { 7.2, 1.6, 4.4, 3.6 }  
};
```

```
float a = matrix[1][2];  
float b = matrix[1,2];
```



RECALL: FUNCTION

Functions are used to break large and complex computing projects into smaller ones such that different people can work on smaller functions at the same time.

Functions usually accept some inputs, processes those inputs and returns an output.



RECALL: CONDITIONAL STATEMENTS

With conditional statements, where program control moves to is dependent on if some predetermined conditions are satisfied or not.



RECURSION

Recursion is a process in which a function indirectly and directly calls itself within its own function block.

Since a function is involved, recursion also allows us to break down a problem into one or more sub-problems that are similar in form to the original problem



RECURSION: EXAMPLE

Suppose we need to add up all of the numbers in an array. Below is an example of a recursive function

```
74  int addNumArray(int arr[], int count) {  
75      if (count == 0) {  
76          return 0;  
77      }  
78      return arr[count - 1] + addNumArray(arr, count - 1);  
79  }
```



RECURSION: EXAMPLE

The function above has two components

- A base case. This handles the case where there are no numbers to add.
- A recursive case that breaks the problem down into a smaller version of the original problem together with an addition.

```
74  int addNumArray(int arr[], int count) {  
75      if (count == 0) {  
76          return 0;  
77      }  
78      return arr[count - 1] + addNumArray(arr, count - 1);  
79  }
```



SUMMARY

- An array is a type of data structure that can store a fixed-size sequential collection of elements of the same type.
- A multi-dimensional array is defined using multiple adjacent square brackets, and the elements of the array may be initialised with values enclosed in curly braces.
- Recursion is a process in which a function indirectly and directly calls itself within its own function block.



FURTHER READING RESOURCES

- Kernighan, B. W., & Ritchie, D. M. (2002). The C programming language.
- Bailey, T. (2005). An introduction to the c programming language and software design. July-2005
- CMSC 202 Lecture Notes: Recursion.
<https://redirect.cs.umbc.edu/~chang/cs202.f98/readings/recursion.html>



DEPARTMENT OF
COMPUTER SCIENCE



THANK
YOU